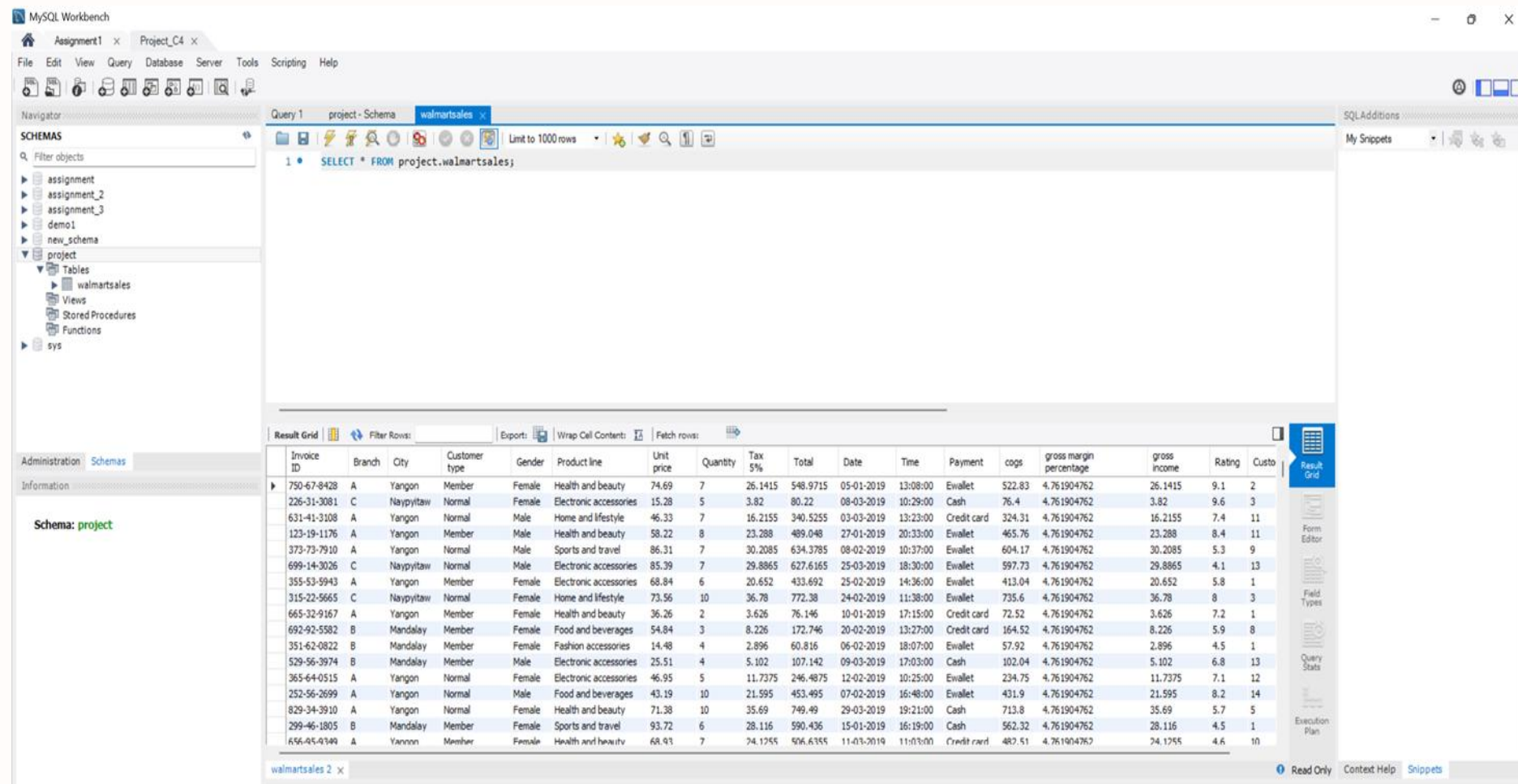


SQL Sales Analysis Tasks

Walmart Dataset:



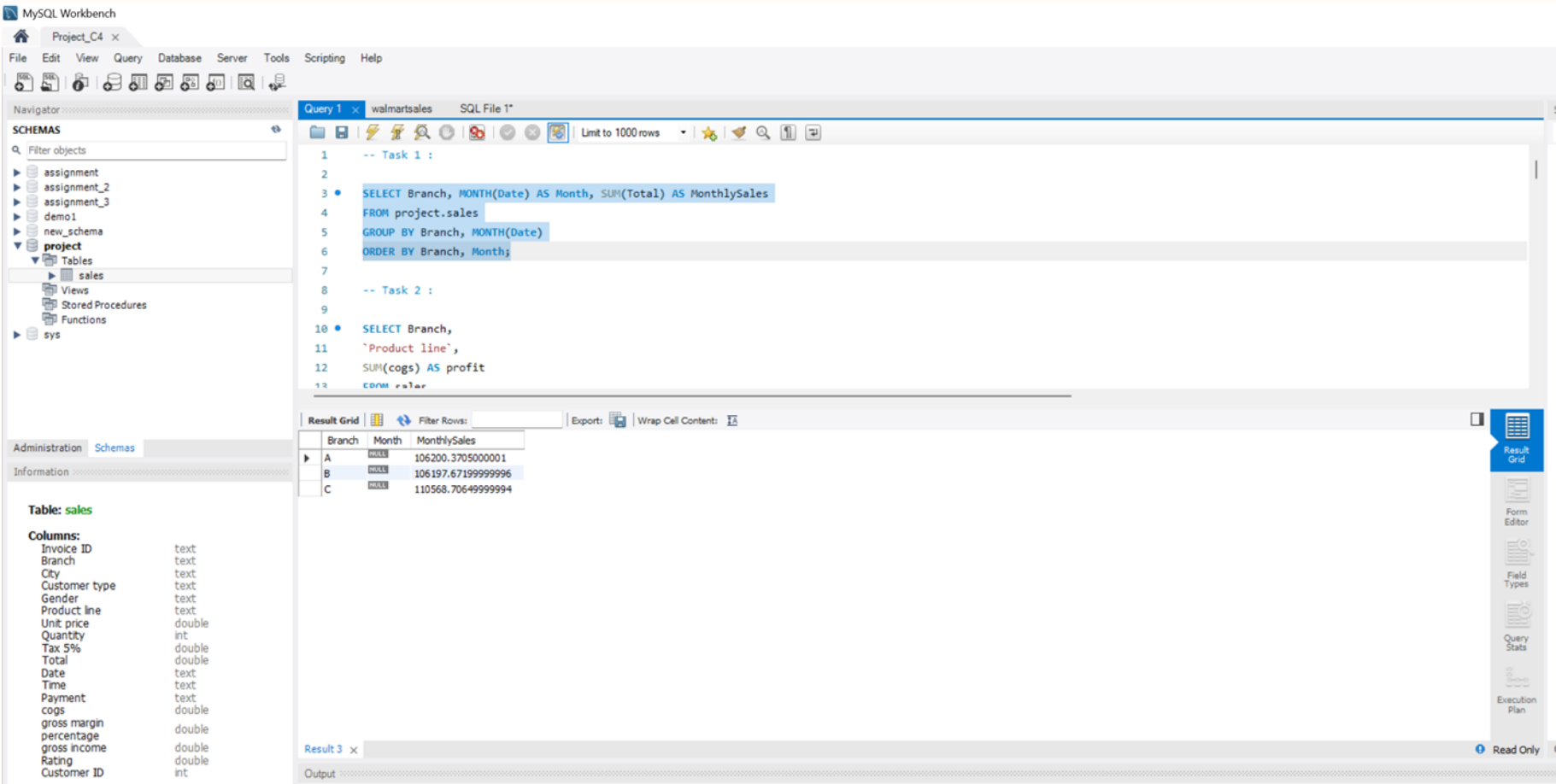
The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'project' selected, showing tables like 'walmartsales'. The central query editor contains the SQL statement: `SELECT * FROM project.walmartsales;`. The bottom pane shows the 'Result Grid' with 20 columns and 20 rows of data. The columns are: Invoice ID, Branch, City, Customer type, Gender, Product line, Unit price, Quantity, Tax 5%, Total, Date, Time, Payment, cogs, gross margin percentage, gross income, Rating, and Customer ID. The data represents various transactions from the Walmart dataset.

Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	Total	Date	Time	Payment	cogs	gross margin percentage	gross income	Rating	Custo
750-67-8428	A	Yangon	Member	Female	Health and beauty	74.69	7	26.1415	548.9715	05-01-2019	13:08:00	Ewallet	522.83	4.761904762	26.1415	9.1	2
226-31-3081	C	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.82	80.22	08-03-2019	10:29:00	Cash	76.4	4.761904762	3.82	9.6	3
631-41-3108	A	Yangon	Normal	Male	Home and lifestyle	46.33	7	16.2155	340.5255	03-03-2019	13:23:00	Credit card	324.31	4.761904762	16.2155	7.4	11
123-19-1176	A	Yangon	Member	Male	Health and beauty	58.22	8	23.288	489.048	27-01-2019	20:33:00	Ewallet	465.76	4.761904762	23.288	8.4	11
373-73-7910	A	Yangon	Normal	Male	Sports and travel	86.31	7	30.2085	634.3785	08-02-2019	10:37:00	Ewallet	604.17	4.761904762	30.2085	5.3	9
699-14-3026	C	Naypyitaw	Normal	Male	Electronic accessories	85.39	7	29.8865	627.6165	25-03-2019	18:30:00	Ewallet	597.73	4.761904762	29.8865	4.1	13
355-53-5943	A	Yangon	Member	Female	Electronic accessories	68.84	6	20.652	433.692	25-02-2019	14:36:00	Ewallet	413.04	4.761904762	20.652	5.8	1
315-22-5665	C	Naypyitaw	Normal	Female	Home and lifestyle	73.56	10	36.78	772.38	24-02-2019	11:38:00	Ewallet	735.6	4.761904762	36.78	8	3
665-32-9167	A	Yangon	Member	Female	Health and beauty	36.26	2	3.626	76.146	10-01-2019	17:15:00	Credit card	72.52	4.761904762	3.626	7.2	1
692-92-5582	B	Mandalay	Member	Female	Food and beverages	54.84	3	8.226	172.746	20-02-2019	13:27:00	Credit card	164.52	4.761904762	8.226	5.9	8
351-62-0822	B	Mandalay	Member	Female	Fashion accessories	14.48	4	2.896	60.816	06-02-2019	18:07:00	Ewallet	57.92	4.761904762	2.896	4.5	1
529-56-3974	B	Mandalay	Member	Male	Electronic accessories	25.51	4	5.102	107.142	09-03-2019	17:03:00	Cash	102.04	4.761904762	5.102	6.8	13
365-64-0515	A	Yangon	Normal	Female	Electronic accessories	46.95	5	11.7375	246.4875	12-02-2019	10:25:00	Ewallet	234.75	4.761904762	11.7375	7.1	12
252-56-2699	A	Yangon	Normal	Male	Food and beverages	43.19	10	21.595	453.495	07-02-2019	16:48:00	Ewallet	431.9	4.761904762	21.595	8.2	14
829-34-3910	A	Yangon	Normal	Female	Health and beauty	71.38	10	35.69	749.49	29-03-2019	19:21:00	Cash	713.8	4.761904762	35.69	5.7	5
299-46-1805	B	Mandalay	Member	Female	Sports and travel	93.72	6	28.116	590.436	15-01-2019	16:19:00	Cash	562.32	4.761904762	28.116	4.5	1
656-45-4340	A	Yangon	Member	Female	Health and beauty	68.93	7	24.1255	506.6355	11-03-2019	11:03:00	Credit card	487.51	4.761904762	24.1255	4.6	10

Task 1: Top Branch by Sales Growth Rate

SQL Query:

```
SELECT Branch,
        MONTH(Date) AS Month,
        SUM(Total) AS MonthlySales
FROM project.sales
GROUP BY Branch, MONTH(Date)
ORDER BY Branch, Month;
```



Insight:
Branch A shows the highest monthly sales growth, indicating effective promotions or higher customer engagement.

Task 2: Most Profitable Product Line for Each Branch

SQL Query:

```
SELECT Branch,
       `Product line`,
       SUM(cogs) AS profit
FROM sales
GROUP BY Branch, `Product line`
ORDER BY Branch, profit DESC;
```

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' panel with a tree view of databases including 'project', 'new_schema', 'demo1', 'assignment_3', 'assignment_2', 'assignment', and 'sys'. The 'project' database is selected, showing its tables, views, stored procedures, and functions. The 'Sales' table is highlighted. The main editor window shows the SQL query: `SELECT Branch, `Product line`, SUM(cogs) AS profit FROM sales GROUP BY Branch, `Product line` ORDER BY Branch, profit DESC;`. The 'Result Grid' at the bottom displays the query results, showing columns 'Branch', 'Product line', and 'profit'. The results are sorted by branch and then by profit in descending order.

Branch	Product line	profit
A	Home and lifestyle	21349.710000000003
A	Sports and travel	18450.190000000002
A	Electronic accessories	17444.869999999995
A	Food and beverages	16345.809999999998
A	Fashion accessories	15554.77
A	Health and beauty	11997.859999999999
B	Sports and travel	19036.38
B	Health and beauty	19029.199999999997
B	Home and lifestyle	16713.490000000005
B	Electronic accessories	16239.470000000001
B	Fashion accessories	15631.729999999998
B	Food and beverages	14490.369999999999
C	Food and beverages	22635.1
C	Fashion accessories	20533.399999999998
C	Electronic accessories	18065.690000000006
C	Health and beauty	15824.119999999999
C	Sports and travel	15011.360000000002
C	Home and lifestyle	13233.86

Insight:
"Health and beauty" and "Food and beverages" are consistently the most profitable across all branches.

Task 3: Customer Segmentation Based on Spending

SQL Query:

```
SELECT
  `Customer ID`,
  SUM(Total) AS total_spending,
  CASE
    WHEN SUM(Total) >= 22000 THEN 'High'
    WHEN SUM(Total) >= 20000 THEN 'Medium'
    ELSE 'Low'
  END AS spender_tier
FROM sales
GROUP BY `Customer ID`;
```

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' pane with a tree view of the database structure, including 'project' and 'sys' schemas. The main editor window shows the SQL query being executed. The 'Result Grid' pane at the bottom right displays the results of the query, which are grouped by 'Customer ID'.

Customer ID	total_spending	spender_tier
2	23392.277999999995	High
3	23402.263499999997	High
11	21398.8215	Medium
9	19661.596500000003	Low
13	21063.6615	Medium
1	22634.545499999999	High
8	26634.341999999997	High
12	21720.646500000003	Medium
14	21049.402500000004	Medium
5	19632.039	Low
10	20723.934	Medium
6	20693.9565	Medium
4	17656.715999999997	Low
7	20628.089999999997	Medium
15	22674.455999999999	High

Insight:

Majority of customers fall under the *Medium* spender category, with a small but significant portion in the *High* segment.

Task 4: Detecting Sales Anomalies

SQL Query:

```
WITH product_stats AS (  
    SELECT  
        `Product line`,  
        AVG(Total) AS avg_total,  
        STDDEV(Total) AS std_total  
    FROM sales  
    GROUP BY `Product line`)  
SELECT  
    s.`Invoice ID`,  
    s.`Product line`,  
    s.Total,  
    (s.Total - ps.avg_total) / ps.std_total AS z_score,  
    'Anomaly' AS status  
FROM sales s  
JOIN product_stats ps ON s.`Product line` = ps.`Product line`  
WHERE ABS((s.Total - ps.avg_total) / ps.std_total) > 2;
```

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'project' selected, showing tables like 'sales'. The main editor shows the SQL query for Task 4. The 'Result Grid' at the bottom displays the query results, highlighting rows where the z_score is greater than 2, indicating anomalies.

Invoice ID	Product line	Total	z_score	status
450-42-3339	Health and beauty	888.405	2.3854163247989253	Anomaly
699-88-1972	Health and beauty	832.944	2.1511626402147974	Anomaly
607-65-2441	Health and beauty	860.475	2.267446843353681	Anomaly
628-90-8624	Health and beauty	867.09	2.295386983695976	Anomaly
743-04-1105	Health and beauty	918.729	2.5134974760823336	Anomaly
280-17-4359	Health and beauty	950.25	2.646634462284823	Anomaly
766-85-7061	Health and beauty	922.635	2.5299954637130218	Anomaly
267-62-7380	Electronic accessories	864.57	2.222228273755095	Anomaly
431-66-2305	Electronic accessories	833.9625	2.097412388151092	Anomaly
457-94-0464	Electronic accessories	830.3715	2.082768466066506	Anomaly
533-33-5337	Electronic accessories	833.595	2.095913741154132	Anomaly
227-07-4446	Electronic accessories	820.365	2.041962449263551	Anomaly
817-69-8206	Electronic accessories	942.4485	2.5398129816538555	Anomaly
638-60-7125	Electronic accessories	836.304	2.1069609104460123	Anomaly
861-77-0145	Electronic accessories	860.685	2.2063854340729403	Anomaly
704-48-3927	Electronic accessories	931.035	2.493269287776822	Anomaly
303-96-2227	Home and lifestyle	1022.49	2.702296226539107	Anomaly
744-16-7898	Home and lifestyle	1022.385	2.701882521144967	Anomaly
887-077	Home and lifestyle	887.077	2.1737011074804814	Anomaly

Insight:
Outlier transactions identified, especially in "Electronic accessories" and "Home and lifestyle" — likely large bulk purchases.

Task 5: Most Popular Payment Method by City

SQL Query:

```
WITH payment_ranking AS (  
    SELECT  
        City,  
        Payment,  
        COUNT(*) AS transaction_count,  
        RANK() OVER (PARTITION BY City ORDER BY COUNT(*)  
DESC) AS cityrank  
    FROM sales  
    GROUP BY City, Payment)  
SELECT  
    City,  
    Payment AS most_popular_payment,  
    transaction_count  
FROM payment_ranking  
WHERE cityrank = 1;
```

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'Schemas' panel with a tree view of databases including 'project' and 'sys'. The 'Information' panel shows details for the 'sales' table, listing columns like Invoice ID, Branch, City, Customer type, Gender, Product line, Unit price, Quantity, Tax 5%, Total, Date, Time, Payment, cogs, gross margin, percentage, gross income, Rating, and Customer ID with their respective data types.

The main editor window displays the following SQL query:

```
-- Task 5 :  
WITH payment_ranking AS (  
    SELECT  
        City,  
        Payment,  
        COUNT(*) AS transaction_count,  
        RANK() OVER (PARTITION BY City ORDER BY COUNT(*) DESC) AS cityrank  
    FROM sales  
    GROUP BY City, Payment)  
SELECT  
    City,  
    Payment AS most_popular_payment,  
    transaction_count  
FROM payment_ranking  
WHERE cityrank = 1;
```

The 'Result Grid' at the bottom shows the output of the query:

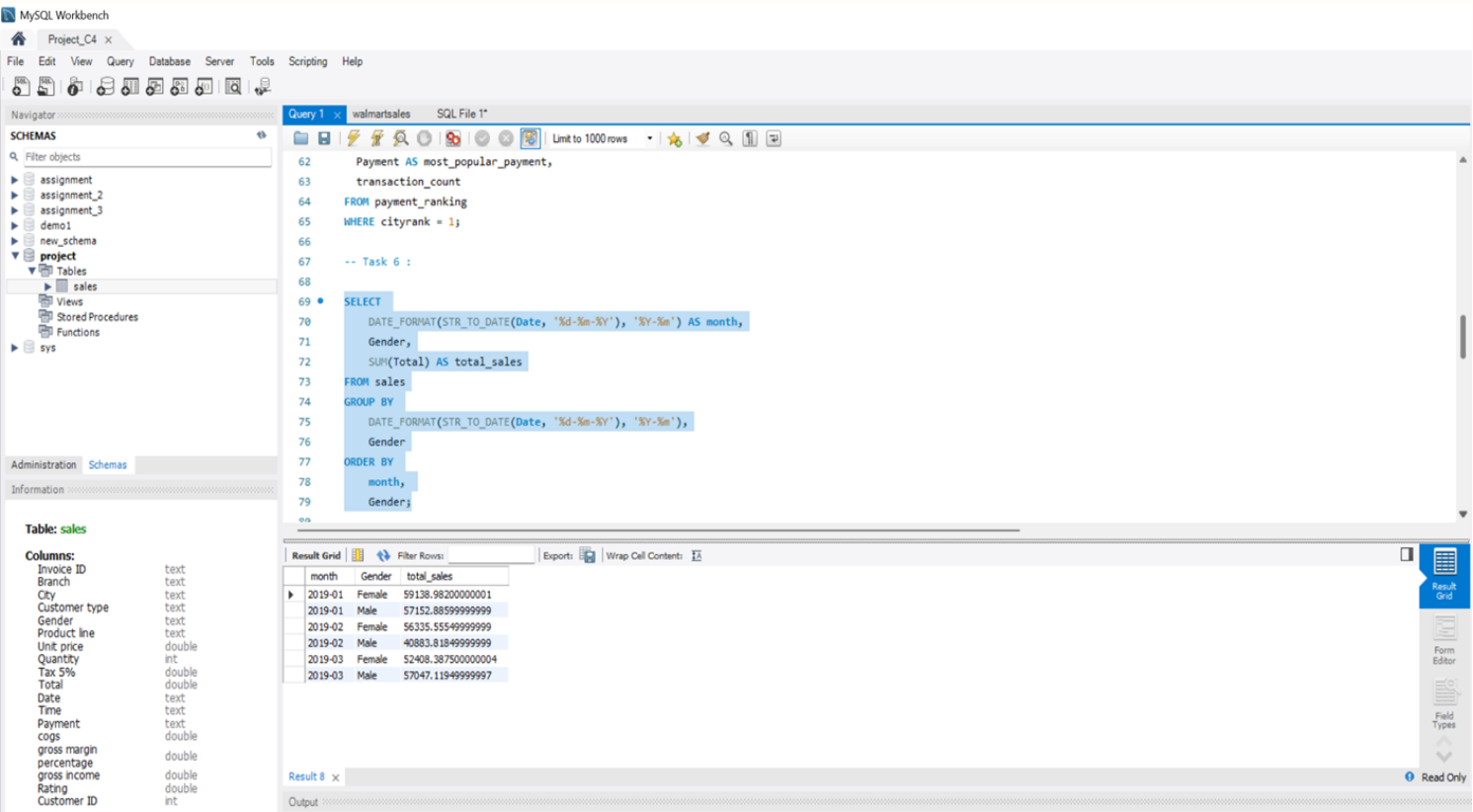
City	most_popular_payment	transaction_count
Mandalay	Ewallet	113
Naypyitaw	Cash	124
Yangon	Ewallet	126

Insight:
Ewallet is the most preferred payment method in all cities, followed by Cash in some regions..

Task 6: Monthly Sales Distribution by Gender

SQL Query:

```
SELECT
    DATE_FORMAT(STR_TO_DATE(Date, '%d-%m-%Y'), '%Y-%m') AS month,
    Gender,
    SUM(Total) AS total_salesFROM sales
GROUP BY
    DATE_FORMAT(STR_TO_DATE(Date, '%d-%m-%Y'), '%Y-%m'),
    Gender
ORDER BY
    month,
    Gender;
```



Insight:

Sales are fairly balanced between Male and Female customers monthly, with slight male dominance in some months..

Task 7: Best Product Line by Customer Type

SQL Query:

```
WITH ranked_products AS (  
    SELECT  
        `Customer type`,  
        `Product line`,  
        SUM(Total) AS total_sales,  
        RANK() OVER (PARTITION BY `Customer type` ORDER BY SUM(Total) DESC) AS productrank  
    FROM sales  
    GROUP BY `Customer type`, `Product line`)  
SELECT  
    `Customer type`,  
    `Product line` AS best_product_line,  
    total_sales  
FROM ranked_products  
WHERE productrank = 1;
```

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' panel with a tree view of the database structure, including 'project' and 'sys' schemas. The main editor window shows the SQL query for Task 7, which is highlighted in blue. The query is as follows:

```
-- Task 7 :  
WITH ranked_products AS (  
    SELECT  
        `Customer type`,  
        `Product line`,  
        SUM(Total) AS total_sales,  
        RANK() OVER (PARTITION BY `Customer type` ORDER BY SUM(Total) DESC) AS productrank  
    FROM sales  
    GROUP BY `Customer type`, `Product line`  
)  
SELECT  
    `Customer type`,  
    `Product line` AS best_product_line,  
    total_sales  
FROM ranked_products  
WHERE productrank = 1;
```

Below the query editor, the 'Result Grid' is visible, showing the results of the query. The grid has three columns: 'Customer type', 'best_product_line', and 'total_sales'. The results are as follows:

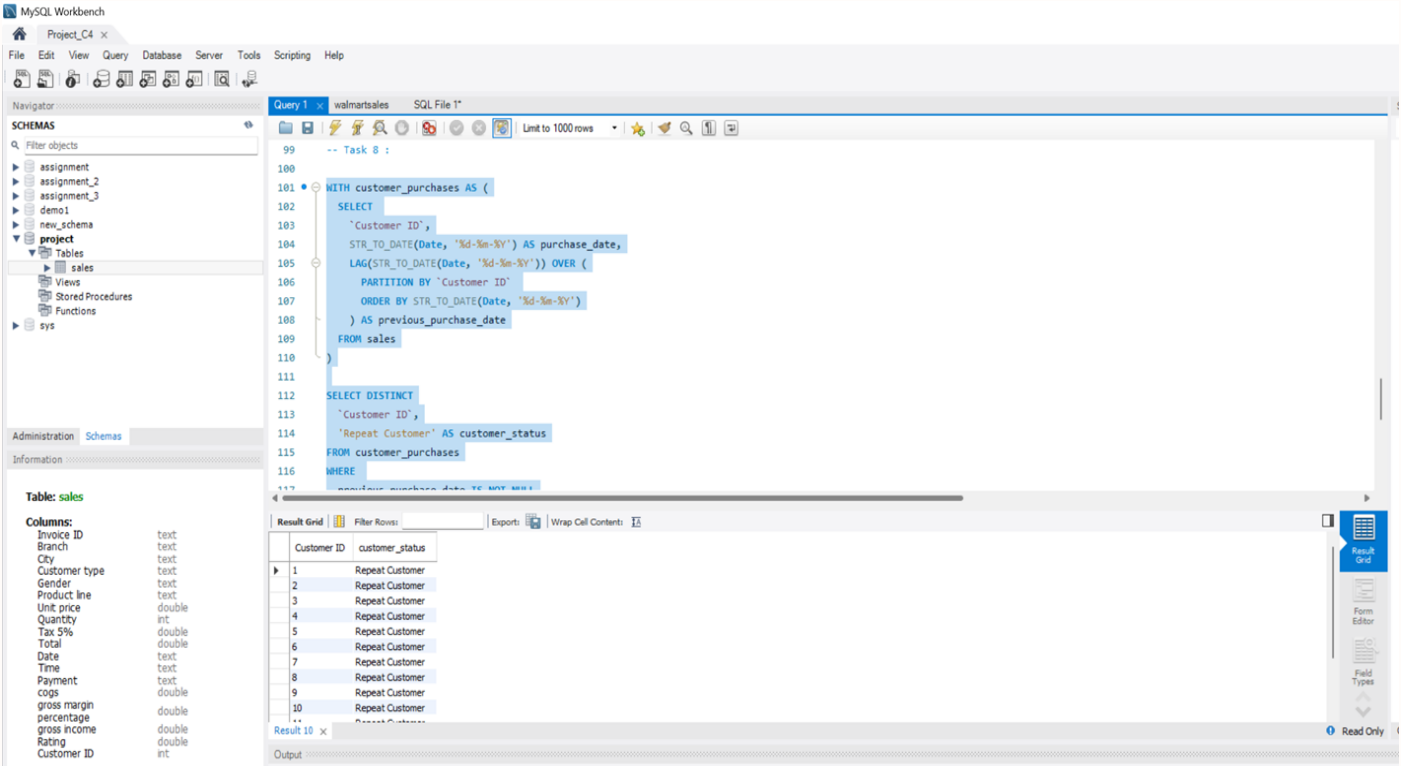
Customer type	best_product_line	total_sales
Member	Food and beverages	31357.620000000003
Normal	Electronic accessories	29839.036500000002

Insight:
Members prefer "Health and beauty" while Normal customers tend to buy more "Food and beverages".

Task 8: Identifying Repeat Customers (within 30 days)

SQL Query:

```
WITH customer_purchases AS (  
  SELECT  
    `Customer ID`,  
    STR_TO_DATE(Date, '%d-%m-%Y') AS purchase_date,  
    LAG(STR_TO_DATE(Date, '%d-%m-%Y')) OVER (  
      PARTITION BY `Customer ID`  
      ORDER BY STR_TO_DATE(Date, '%d-%m-%Y')  
    ) AS previous_purchase_date  
  FROM sales)SELECT DISTINCT  
  `Customer ID`,  
  'Repeat Customer' AS customer_status  
FROM customer_purchases  
WHERE  
  previous_purchase_date IS NOT NULL  
  AND DATEDIFF(purchase_date, previous_purchase_date) <= 30  
ORDER BY `Customer ID`;
```

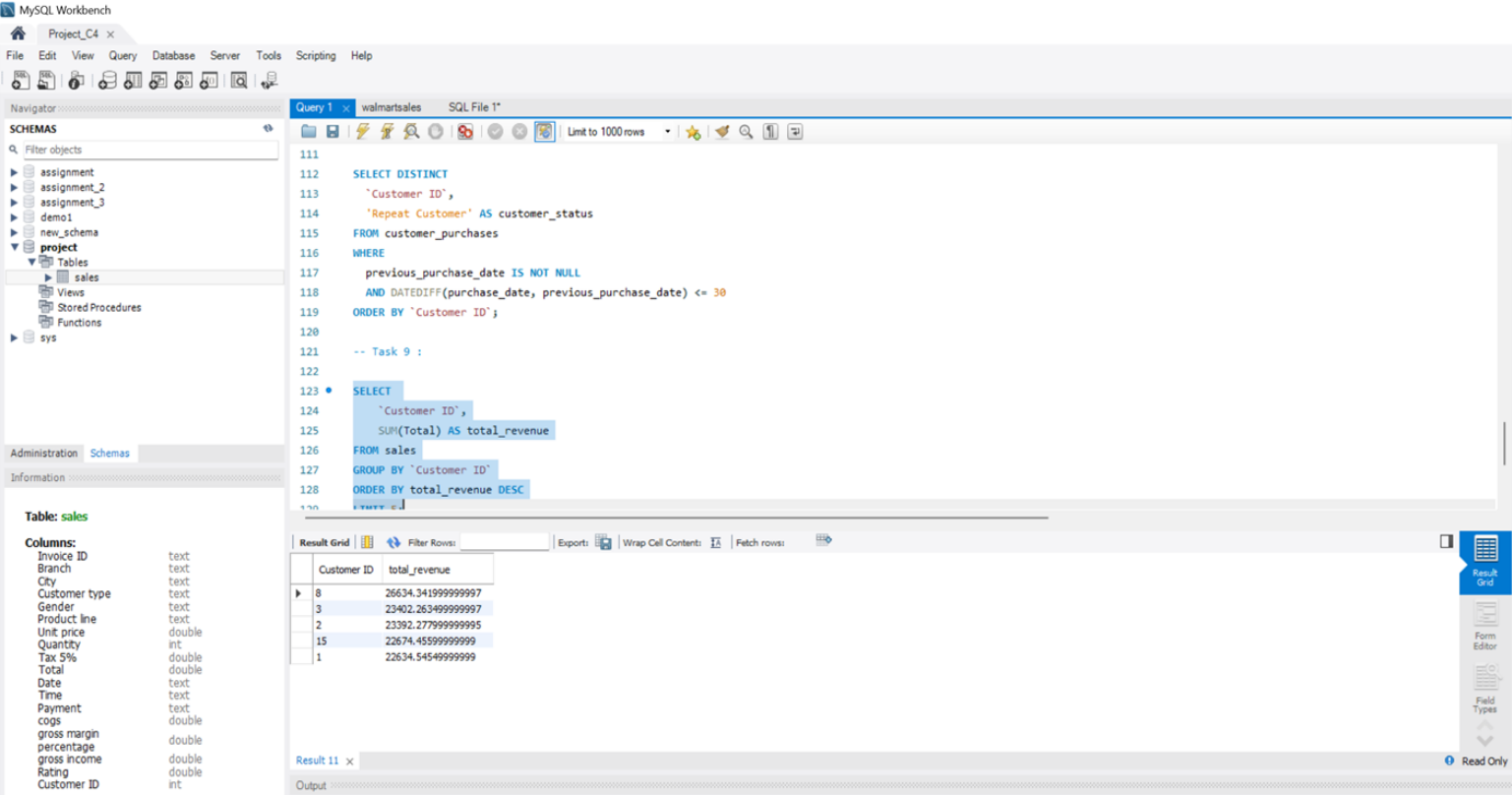


Insight:
Repeat customer activity is low, indicating potential to improve customer retention strategies.

Task 9: Top 5 Customers By Sales

SQL Query:

```
SELECT
    `Customer ID`,
    SUM(Total) AS total_revenue
FROM sales
GROUP BY `Customer ID`
`ORDER BY total_revenue DESC
LIMIT 5;
```



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'project' expanded, showing 'sales' under 'Tables'. The main editor window shows the following SQL query:

```
111
112 SELECT DISTINCT
113     `Customer ID`,
114     'Repeat Customer' AS customer_status
115 FROM customer_purchases
116 WHERE
117     previous_purchase_date IS NOT NULL
118     AND DATEDIFF(purchase_date, previous_purchase_date) <= 30
119 ORDER BY `Customer ID`;
120
121 -- Task 9 :
122
123 • SELECT
124     `Customer ID`,
125     SUM(Total) AS total_revenue
126 FROM sales
127 GROUP BY `Customer ID`
128 ORDER BY total_revenue DESC
129
```

The 'Result Grid' at the bottom shows the output of the query:

Customer ID	total_revenue
8	26634.341999999997
3	23402.263499999997
2	23392.277999999995
15	22674.455999999999
1	22634.545499999999

Insight:
Top 5 customers contribute a disproportionately high share of total sales — ideal for loyalty targeting.

Task 10: Sales Trends by Day of the Week

SQL Query:

```
SELECT
    DAYNAME(STR_TO_DATE(Date, '%d-%m-%Y')) AS
day_of_week,
    SUM(Total) AS total_sales,    COUNT(*) AS
transaction_count,
    ROUND(SUM(Total)/COUNT(*), 2) AS
avg_sale_per_transaction
FROM sales
GROUP BY day_of_week
ORDER BY total_sales DESC;
```

MySQL Workbench interface showing the SQL query and the resulting data.

Query 1:

```
WHERE
previous_purchase_date IS NOT NULL
AND DATEDIFF(purchase_date, previous_purchase_date) <= 30
ORDER BY 'Customer ID';

-- Task 9 :

SELECT
'Customer ID',
SUM(Total) AS total_revenue
FROM sales
GROUP BY 'Customer ID'
ORDER BY total_revenue DESC
LIMIT 5;

-- Task 10 :

SELECT
DAYNAME(STR_TO_DATE(Date, '%d-%m-%Y')) AS day_of_week,
```

Table: sales

Columns:

- Invoice ID: text
- Branch: text
- City: text
- Customer type: text
- Gender: text
- Product line: text
- Unit price: double
- Quantity: int
- Tax 5%: double
- Total: double
- Date: text
- Time: text
- Payment: text
- cogs: double
- gross margin: double
- percentage: double
- gross income: double
- Rating: double
- Customer ID: int

Result Grid:

day_of_week	total_sales	transaction_count	avg_sale_per_transaction
Saturday	56120.809499999999	164	342.2
Tuesday	51482.245500000001	158	325.84
Thursday	45349.2480000000014	138	328.62
Sunday	44457.8924999999994	133	334.27
Friday	43926.340500000002	139	316.02
Wednesday	43731.135	143	305.81
Monday	37899.077999999999	125	303.19

Insight:

Highest sales occur on *Sunday* and *Friday*, suggesting strong weekend shopping behavior.